

Spring AI 첫걸음

ChatClient로 Gemini와 대화하는 백엔드 만들기

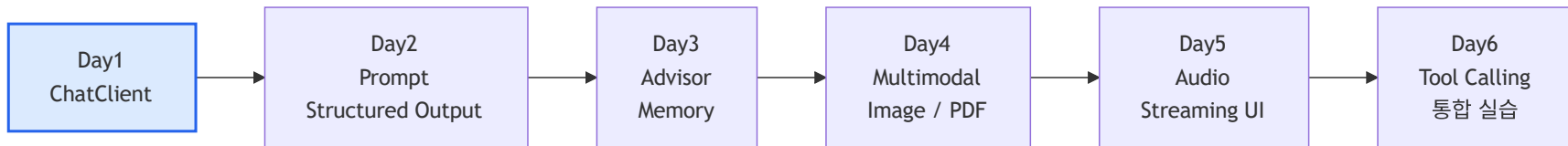
Day1 · 생성형 AI를 Spring Boot 백엔드에 붙이는 첫 실습



오늘 목표

1. AI 응용 서비스 구조(Front-End↔Back-End↔AI 모델)를 설명할 수 있다
2. `spring-ai-starter-model-google-genai` + `GOOGLE_API_KEY` 연결
3. 자동구성된 `ChatClient` 로 Gemini 응답 엔드포인트 구현
4. API Key를 서버 환경변수로만 관리해야 하는 이유
5. RPM/TPM/RPD를 직접 확인하고 모델을 고르는 이유
6. `.system(...)` 으로 역할을 부여하고 모델 옵션을 관찰
7. 미니 도구 2개 이상 완성 + GitHub push

오늘의 위치



1. Spring AI 수업의 목표

지금까지의 흐름과 오늘의 흐름은 뭐가 다를까?

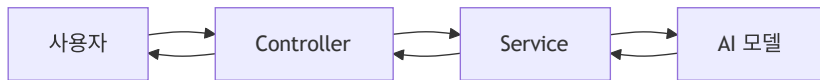
CRUD 흐름 vs AI 흐름

지금까지 만든 것



"DB에 물어보고 응답"

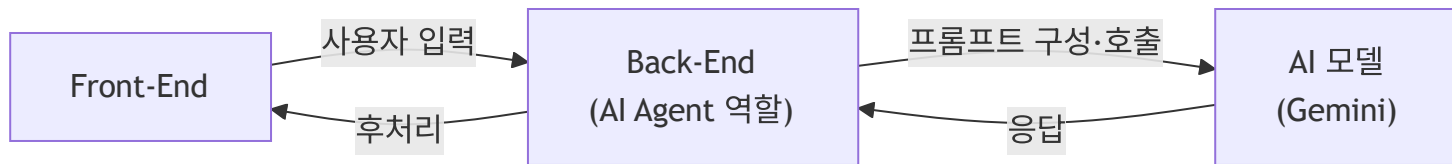
오늘 만드는 것



"LLM에 물어보고 응답"

묻는 대상만 DB → LLM으로 바뀌었을 뿐, Controller가 요청을 받아 처리하는 큰 틀은 같다

AI 응용 서비스 아키텍처



Back-End의 역할 3가지:

1. Front-End와 상호작용하며 사용자 의도를 이해
2. 입력을 가공·보강해 프롬프트를 구성하고 LLM을 호출
3. 응답을 후처리해 Front-End에 전달

오늘 만드는 `ChatController` → `ChatService` 가 이 구조의 최소 버전이다

2. Spring AI 개요

Spring AI란?

Spring 생태계에서 생성형 AI 모델(OpenAI·Google Gemini·Anthropic 등)을 다루기 위한 **공식** 프레임워크

Spring Data가 여러 DB를 공통 인터페이스로 다루듯, **Spring AI는 여러 AI 프로바이더를 공통 인터페이스 (`ChatClient`)로 다룬다**

파이썬 생태계의 LangChain과 비슷한 역할 — "프롬프트→응답→후처리"를 표준 방식으로 연결한다는 핵심은 같다 (교재 1.3)

왜 필요한가 — 프로바이더마다 다른 스키마

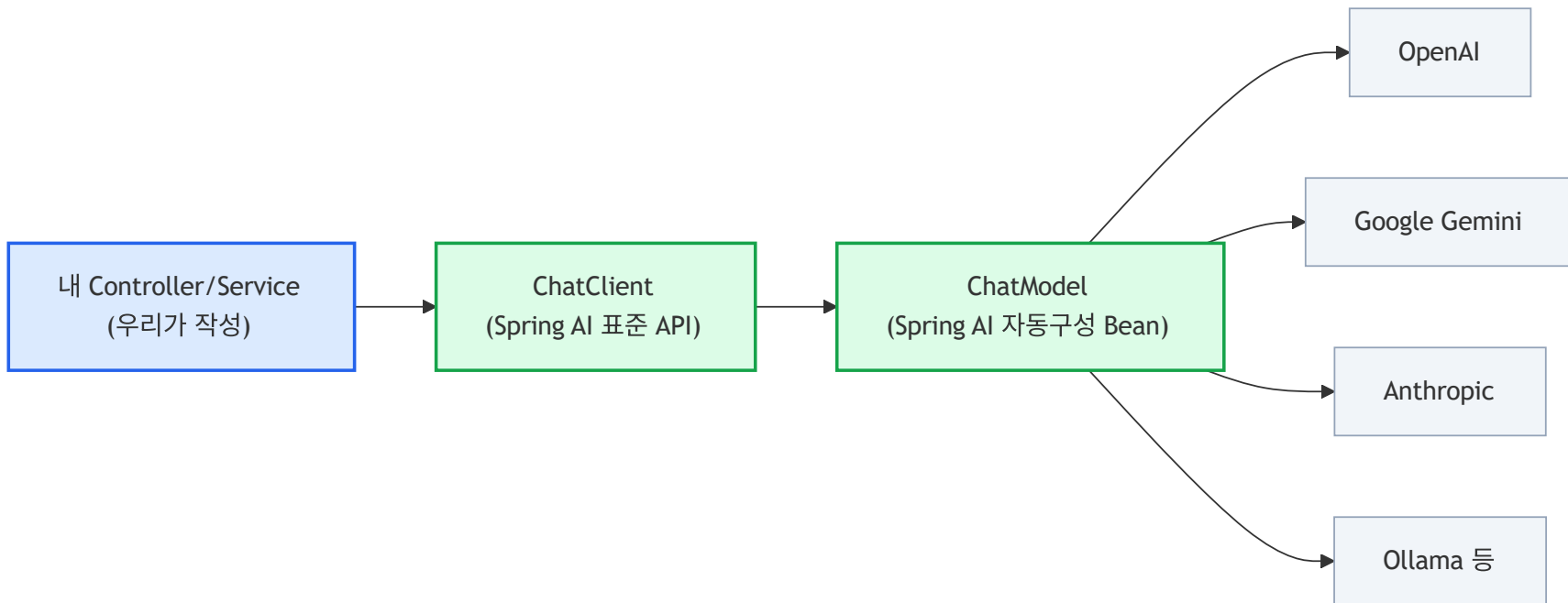
프로바이더마다 REST API의 요청/응답 JSON 스키마가 다르다 — Spring AI가 이 복잡도를 흡수한다

```
// Spring AI 없이 - 프로바이더별 JSON 스키마를 직접 조립
Map<String, Object> body = Map.of(
    "contents", List.of(Map.of("parts", List.of(Map.of("text", message))))
);
ResponseEntity<Map> res = restTemplate.postForEntity(geminiUrl, body, Map.class);
String reply = /* 응답 JSON 구조를 손으로 파싱 */;

// Spring AI로 - 프로바이더가 Gemini든 OpenAI든 코드는 동일
String reply = chatClient.prompt().user(message).call().content();
```

프로바이더를 OpenAI로 바꿔도 `application.yml` 프로퍼티 이름만 바꿀 뿐, 위 두 번째 코드는 그대로 재사용된다

Spring AI가 서 있는 위치



우리 코드는 `ChatClient` 까지만 알면 된다 — 실제 어느 프로바이더로 가는지는 설정(`application.yml`)이 결정한다

모델 유형과 Provider

| 지원 모델 유형

- **Chat** (LLM) ← 오늘 이것만
- Embedding
- Image
- Audio (TTS/STT)

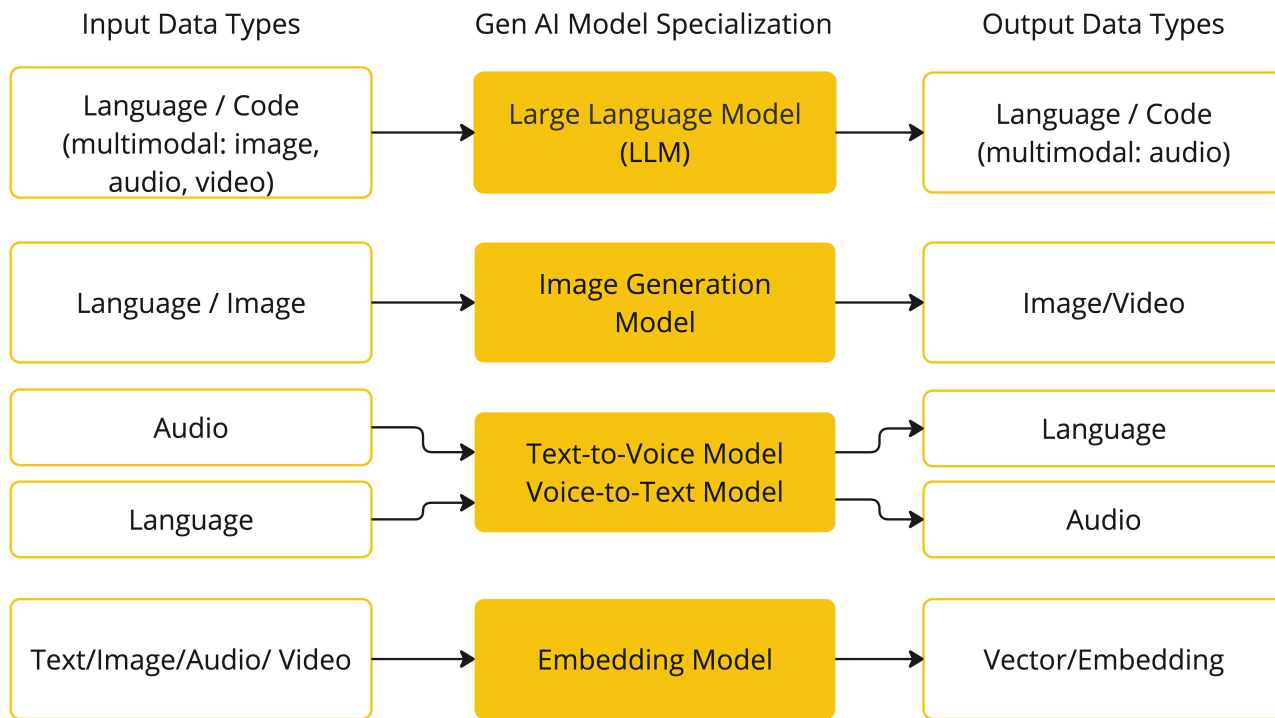
AI 모델 = 아키텍처(구조) + 파라미터(값)

| 주요 Provider

- OpenAI
- **Google (Gemini)** ← 오늘 고정
- Anthropic
- Ollama 등

같은 `ChatClient` 코드로 프로바이더 교체 가능

모델 유형별 입출력 구조



출처: Spring AI 공식 문서 (docs.spring.io)

Provider 비교

Provider	실행 위치	특징	오늘 사용
OpenAI	클라우드	가장 널리 쓰임, 유료 위주	개념만 언급
Google (Gemini)	클라우드	무료 티어 제공	오늘 고정
Anthropic	클라우드	Claude 모델군	개념만 언급
Ollama	로컬(내 PC)	무료, 오프라인 실행 가능	개념만 언급

프로바이더를 바꿔도 우리 코드(`ChatClient` 호출부)는 그대로 — `application.yml` 설정만 바뀐다

ChatModel vs ChatClient — 개요

	ChatModel	ChatClient
성격	저수준 인터페이스	고수준 API (fluent, 체이닝)
입출력	Prompt → ChatResponse	.user(message) → .content()
오늘 실습	개념만 인지	직접 사용

ChatClient 가 내부적으로 ChatModel 을 감싸 쓰기 편하게 만든 것 — 자세한 연결 과정은 잠시 후(6-7단계)에서 다룬다

Spring AI vs LangChain

비교 항목	LangChain	Spring AI
언어/플랫폼	Python	Java / Spring Boot
핵심 개념	프롬프트-응답을 체인(chain)으로 연결	스프링 빈으로 자동 주입된 API 호출
대화 기억	지원	지원 (Day4)
생태계 통합	Python AI 생태계	Spring Boot 생태계(DI·자동구성)

핵심은 같다 — "프롬프트 → 응답 → 후처리"를 표준화된 방식으로 연결한다

3. 버전과 개발 환경

오늘의 버전

항목	버전
JDK 	21 LTS
Spring Boot 	4.1.x
Spring AI	2.0.0 GA
Provider 	Google GenAI (Gemini)

⚠️ JDK ≠ JRE

`javac -version` 이 안 뜨면 JRE만 설치된 것 —
Gradle 빌드가 `JAVA_COMPILER` 오류로 실패한다

Spring AI **2.0.0**(GA)과 `2.0.0-M8` (마일스톤)은 다르다 — 구버전 예제를 그대로 베끼지 않는다

4-5. 프로젝트 생성 + Provider 설정

Spring Initializr

항목	값
Project	Gradle
Group	com.example
Artifact / Name	day01-chat-client
Package name	com.example.day01chatclient
Java	21
Dependencies	Spring Web, Google GenAI

전원 동일한 출발선에서 시작 — 기존 CRUD 프로젝트에 이어서 작업하지 않는다

application.yml 설정

```
spring:
  application:
    name: day01-chat-client
  ai:
    google:
      genai:
        api-key: ${GOOGLE_API_KEY}
        chat:
          model: gemini-3.1-flash-lite
          temperature: 0.7
```

Spring AI가 `GOOGLE_API_KEY` 를 자동으로 읽어주는 게 아니다 — `${...}` 로 명시적으로 연결해야 한다

OpenAI였다면? (개념 비교만)

프로바이더를 바꿔도 코드는 그대로, 설정 경로만 바뀐다

Gemini (오늘)

```
spring:
  ai:
    google:
      genai:
        api-key: ${GOOGLE_API_KEY}
        chat:
          model: gemini-3.1-flash-lite
```

OpenAI (실습 안 함)

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      chat:
        options:
          model: gpt-4o-mini
```

`ChatClient` 코드는 한 줄도 안 바뀐다 — 바뀌는 건 의존성과 설정 경로뿐

토큰이란? "토큰 = 돈"

- **토큰(token)** = LLM이 텍스트를 처리하는 최소 단위. 단어 하나가 그대로 1토큰이 되기도, 긴 단어·한글은 여러 조각(subword)으로 쪼개지기도 한다
- 토큰은 **입력**(내가 보낸 질문)과 **출력**(모델의 답변) **모두**에 걸쳐 과금·한도에 영향을 준다
- "무료" ≠ "무제한" — 무료 티어에도 토큰 수·요청 수 한도가 있다

LLMs like ChatGPT generate tokens, not words. Although tokens often end up being complete words, understanding the distinction is essential for understanding how LLM settings affect their outputs, why oddities like universal adversarial triggers occur, how AI-text detectors work, etc.

출처: Spring AI 공식 문서 (docs.spring.io)

RPM 분당 요청 수

TPM 분당 토큰 수

RPD 하루 요청 수

무료 티어 한도 비교

모델	RPM	RPD
gemini-2.5-flash	5	20
gemini-2.5-flash-lite	10	20
gemini-3-flash	5	20
gemini-3.5-flash	5	20
gemini-3.1-flash-lite	15	500

수업 당일 아침 AI Studio에서 재확인 (계정/정책에 따라 변동)

gemini-3.1-flash-lite

실측 RPD 500 — 반 전체 실습에 안전

오해 주의: 공식 문서의 500 RPD 는 대부분 Search/Maps Grounding(도구 호출) 한도다. 일반 채팅 호출 (generateContent) 한도가 아니다 — 위 표는 AI Studio 화면에서 직접 확인한 실측치다.

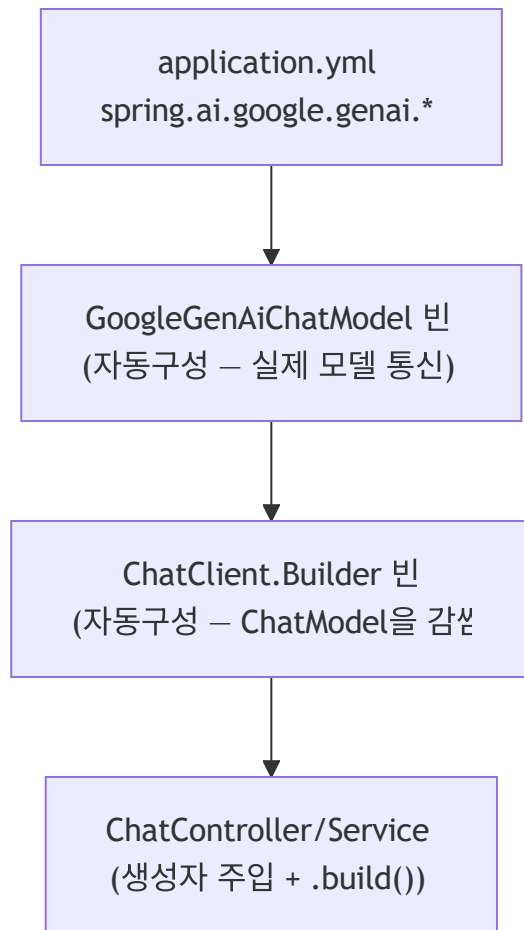
6-7. ChatClient 첫 실습

ChatClient란?

- AI 모델에게 "역할 부여 → 질문 → 호출 → 결과 추출"을 메서드 체이닝(fluent API)으로 표현한 Spring AI의 핵심 인터페이스
- `spring-ai-starter-model-google-genai` 의존성만 추가하면 Spring Boot가 `ChatClient.Builder` 빈을 자동구성해준다 — 직접 빈 등록 코드 불필요
- `Builder` 는 **prototype scope** — 주입될 때마다 새 인스턴스, 그래서 `.build()` 로 실제 사용할 `ChatClient` 를 만든다

```
chatClient.prompt() // 1. 프롬프트 시작
    .system("...") // 2. 역할 부여 (선택)
    .user(message) // 3. 질문
    .call()         // 4. 호출
    .content();      // 5. 결과(텍스트) 추출
```


AI 모델이 Bean으로 연결되는 과정



첫 ChatClient 엔드포인트

```
@RestController
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    @GetMapping("/api/chat")
    public String chat(@RequestParam String message) {
        return chatClient.prompt()
            .user(message)
            .call()
            .content();
    }
}
```

코드 한 줄씩 짚기

코드	설명
<code>ChatClient.Builder</code>	자동구성 — prototype scope, 별도 <code>@Bean</code> 불필요
<code>.build()</code>	실제 사용할 <code>ChatClient</code> 생성
<code>.prompt()</code>	요청 구성 시작
<code>.user(message)</code>	사용자 질문을 user message로 설정
<code>.call()</code>	동기 호출 (비동기 <code>.stream()</code> 은 Day2)
<code>.content()</code>	응답을 <code>String</code> 으로 반환

질문 전달 흐름: `message` (문자열) → `.user()` → `UserMessage` → `Prompt` → 모델 호출 → 텍스트 응답

실제 호출 확인

```
$ curl "http://localhost:8080/api/chat?message=안녕,+너는+누구야?"
```

저는 구글에서 훈련받은 대규모 언어 모델입니다.

흔한 오류: `GOOGLE_API_KEY` 없이 실행하면 앱은 **정상 기동**하고, **호출 시점에만** `500 + API_KEY_INVALID` 발생
— 시작 실패가 아니라 요청 실패라는 게 핵심

8. Prompt 기본기 (맛보기)

system message로 역할 부여

같은 질문, system 하나로 톤과 구조가 달라진다

`/api/chat` (일반)

DI는 Dependency Injection의 약자로... (담백한 개념 설명)

`/api/teacher` (system 프롬프트)

*안녕하세요! ... 오늘은 Spring의 핵심 중 핵심인 DI에 대해 아주
쉽게 설명해 드릴게요... (예시·주의점 포함, 훨씬 친절)*

같은 모델·같은 질문인데 `system()` 하나로 응답의 성격이 확연히 달라진다

출력 형식 지시 + 나쁜/좋은 프롬프트

형식 지시 없음

AI(Artificial Intelligence)는 한국어로 '인공지능'이라고 합니다...
(헤더·불릿 포함, 긴 설명)

"한 문장으로만"

AI(인공지능)란 인간의 학습, 추론, 지각 능력을 컴퓨터 프로그램
으로 구현하여... (한 문장)

나쁜 프롬프트: "AI가 뭐야?" → 매번 백과사전식 일반론

좋은 프롬프트: "Spring AI의 ChatClient가 하는 역할을 초보 백엔드
개발자에게 3줄 이내로 설명해줘" → 대상·주제·분량이 명시된, 정확
하고 짧은 답

자연어 지시는 강제가 아니라 "부탁"이다 — 확실한 형식 강제는 Day3의 구조화된 출력에서

9. 모델 옵션 맛보기

temperature 란?

LLM은 다음 토큰을 **확률적으로** 골라 응답을 만든다 — temperature 는 그 정도를 조절한다

낮음 (0.0에 가까울수록)

확률 가장 높은 단어 선택 → 안정적·일관된 답
사실 확인, 분류 작업에 유리

높음 (1.0에 가까울수록)

낮은 확률 단어도 종종 선택 → 다양·창의적인 답
아이디어 브레인스토밍에 유리

옵션이 조정하는 것

옵션	의미	오늘 설명 수준
<code>model</code>	사용할 모델명	<code>gemini-3.1-flash-lite</code> 고정 이유
<code>temperature</code>	응답의 다양성	낮으면 안정적, 높으면 다양함
<code>max-output-tokens</code>	출력 길이 제한	너무 긴 응답 방지
<code>candidate-count</code> / <code>n</code>	응답 후보 수	비용과 직결 — 기본값 1 권장

옵션 실험도 호출 횟수에 포함된다 — "토큰 = 돈"과 연결해서 신중하게

10. 미니 실습

오늘 배운 패턴으로 미니 도구 만들기

`.system()` + `.user()` + `.call()` 하나로 실용적인 도구를 만들 수 있다 — Step 8에서 연습한 **역할 부여**·**출력 형식 지시**를 그대로 적용한다

코드 설명 도우미 코드 스니펫 → 설명

README 생성 도우미 프로젝트 설명 → README 초안

면접 질문 생성기 기술 스택 → 예상 면접 질문

기업 FAQ 응답기 자주 묻는 질문 → 답변 초안

4종 중 **최소 2개 완성** + README 정리 + GitHub push

제출 — GitHub push

```
cd day01-chat-client
git init
git add .
git commit -m "day01: ChatClient 첫 실습"
git remote add origin https://github.com/본인계정/day01-chat-client.git
git push -u origin main
```

제출 기준: 코드 + 최소 2개 미니 엔드포인트 응답 캡처 + push 링크. **응답 품질은 채점하지 않는다** — 끝까지 동작시켰는지가 기준

막혔을 때 체크리스트

셀프 진단 먼저, 그다음 버디, 그다음 강사

1. `javac -version` 이 뜨는가?
2. 서버가 "Started ... Application"까지 로그에 찍혔는가?
3. 호출이 500이면 → 로그에 `API_KEY_INVALID` 가 보이는가?
4. `application.yml` 에 탭 문자가 섞이지 않았는가?
5. URL 파라미터 이름이 `@RequestParam` 변수명과 일치하는가?
6. 그래도 안 되면 옆자리 버디 → 강사

오늘의 핵심 질문 — 스스로 답해보기

1. Spring AI란 무엇인가?
2. Spring AI가 왜 필요한가?
3. Spring Boot에서 AI 모델을 어떻게 Bean으로 다루는가?
4. `ChatClient` 로 사용자의 질문을 어떻게 모델에 전달하는가?
5. 간단한 REST API로 AI 응답을 반환하는 백엔드를 어떻게 만드는가?

오늘 배운 것

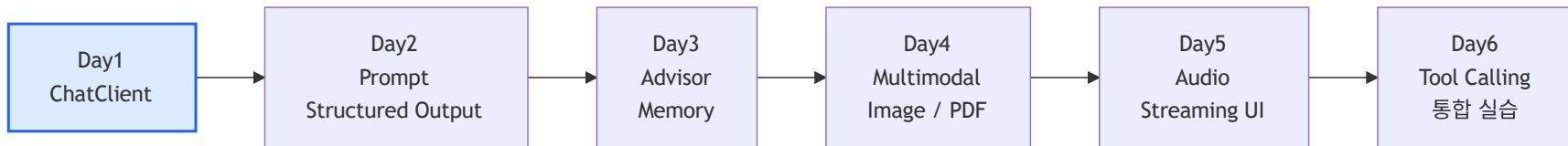
- AI 응용 서비스 아키텍처 (Front-End ↔ Back-End ↔ AI 모델)
- `ChatClient.Builder` 자동구성 주입 — 별도 `@Bean` 불필요
- `user()` / `system()` 메시지 역할 차이
- Controller → Service 계층 분리
- `temperature` 등 모델 옵션 맛보기
- Gemini Free Tier RPM/TPM/RPD 개념
- API Key는 서버 환경변수로만 관리

핵심 용어 정리

용어	정의
ChatClient	AI 모델과 대화하는 fluent API
ChatClient.Builder	자동구성된 prototype bean
ChatModel	더 저수준의 호출 인터페이스
prompt()	요청 구성 시작
system message	모델의 역할·톤을 지정하는 상위 지시
user message	사용자의 실제 질문
call() / content()	동기 호출 / 응답을 문자열로 반환
Provider	Gemini·OpenAI 등 AI 모델 공급자

용어	정의
RPM/TPM/RPD	분당 요청/분당 토큰/일일 요청 한도
temperature	응답 다양성을 조절하는 옵션
candidate-count	응답 후보 개수 옵션
API_KEY_INVALID	키 누락·오류 시 호출 시점 오류
BOM	의존성 버전을 묶어 관리하는 선언
PromptTemplate	파라미터화된 프롬프트 (Day2)
Advisor	호출 흐름의 공통 관심사 처리 (Day3)

6일 로드맵에서 오늘 위치



내일 예고 — Day2

오늘은 AI 응답을 문자열로 받았습니다.

- **PromptTemplate** — 프롬프트를 재사용 가능한 코드 자산으로 만들기
- **구조화된 출력** — 문자열 응답을 Java record/DTO로 받아 업무 로직에 연결하기

오늘의 `.call().content()` 문자열 응답이 내일 `.entity(...)` typed response와 어떻게 다른지가 첫 비교 포인트입니다. Advisor와 Memory는 Day3에서 다룹니다.

수고하셨습니다!

오늘 목표: `ChatClient` 첫 호출 + 미니 도구 2개 + GitHub push